Pretrained Encoders for Classification

Pre-Work Module 02

ECBS5200 — Practical Deep Learning Engineering

What does "pretrained" mean?

A pretrained model has already learned language by reading **massive** amounts of text.

- Billions of words from books, articles, code, web pages
- Trained for days or weeks on expensive hardware
- Learned grammar, syntax, word meaning, context, nuance

You don't train this from scratch. Someone already did.

You download their work and build on top of it.

The encoder architecture

Input: a sequence of token IDs (from Module 01)
Output: a dense vector for every token

```
Token IDs:   [CLS]  I   have  a  billing  problem  [SEP]
                ↓   ↓    ↓    ↓     ↓       ↓       ↓
Encoder:     [ ... transformer layers ... ]
                ↓   ↓    ↓    ↓     ↓       ↓       ↓
Vectors:     [768] [768] [768] [768] [768]  [768]  [768]
```

Each token becomes a **768-dimensional vector** — a rich numerical representation that captures meaning in context.

The [CLS] token: your sentence summary

Remember from Module 01: `[CLS]` is added at the start of every input.

After passing through the encoder, the **[CLS] vector becomes a summary of the entire input**.

```
[CLS] representation → 768 numbers summarizing the whole complaint
```

This is the vector we'll use for classification.

Why [CLS]? Because it has no "word meaning" of its own — the model learns to pack the overall meaning of the input into this position.

The classification head

A **classification head** is a small network on top of the encoder:

```
[CLS] vector (768) → Dense (768→768) → GELU → LayerNorm → Linear (768→111)
```

- A small **prediction head** transforms the [CLS] representation (one dense layer + activation + normalization)
- A final **linear layer** maps to **113 outputs** (one per complaint category, after Week 1 label cleanup)
- The highest score = the model's prediction

~676K parameters total — less than **0.5%** of the model. The encoder does the hard work.

Why not train from scratch?

Training a language model from scratch requires:

- **Billions** of training tokens
- **Hundreds** of GPU-hours (or more)
- **Thousands** of dollars in compute

We have:

- ~220,000 labeled complaints
- Free Kaggle GPUs (30 hrs/week)
- One semester

From-scratch training is not an option. Transfer learning is.

Transfer learning: stand on the shoulders of giants

```
Step 1: Someone trains a model on billions of words    (expensive, done once)
Step 2: You download their pretrained model             (free, takes 30 seconds)
Step 3: You add a classification head                   (a small network)
Step 4: You fine-tune on YOUR data                      (cheap, takes minutes)
```

The model already understands language.

You just teach it: **"given this complaint, which category?"**

This is why modern NLP is accessible. You don't need Google's budget.

ModernBERT-base: our model for the semester

Property	Value
Model	`answerdotai/ModernBERT-base`
Parameters	~149 million
Hidden size	768
Layers	22 transformer layers
Vocabulary	~50,368 tokens
Max sequence length	8,192 tokens (we use 128)

- A modern, efficient encoder released by Answer.AI
- "Base" size — large enough to be powerful, small enough to fine-tune on free GPUs
- This is the backbone we'll use for every experiment

Base model vs. classification model

Two ways to load the same backbone:

Base model (`AutoModel`)
→ Gives you raw embeddings (768-dim vectors for each token)
→ No prediction capability by itself

Classification model (`AutoModelForSequenceClassification`)
→ Same backbone + a classification head
→ Gives you logits (one score per class)

```
# Base: 149M parameters (encoder only)
base = AutoModel.from_pretrained("answerdotai/ModernBERT-base")

# Classification: 149M + ~676K parameters (encoder + head)
classifier = AutoModelForSequenceClassification.from_pretrained(
    "answerdotai/ModernBERT-base", num_labels=111
)
```

Key takeaways

1. **Pretrained models** learned language from billions of words — you get that knowledge for free
2. **Encoders** turn token sequences into dense, context-aware vectors
3. **The [CLS] token** becomes a summary of the entire input
4. **The classification head** is a small network (~676K params) mapping [CLS] → class scores
5. **Transfer learning** lets you build on existing work instead of starting from scratch
6. **ModernBERT-base** (149M params) is our backbone for every experiment this semester

Next: try it yourself in the notebook!